

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO

Môn: Các vấn đề hiện đại của Công nghệ thông tin

**ĐỀ TÀI: SymQEMU: Thực thi biểu tượng cho
tệp nhị phân**

Nhóm 14

Lời mở đầu

Thực thi biểu tượng trừu tượng là một công cụ vượt trội dành cho phân tích phần mềm và tìm kiếm lỗi. Thực thi biểu tượng dựa trên biên dịch là một phương pháp được đề xuất để có thể cải thiện hiệu suất của thực thi biểu tượng một cách đáng kể nếu như bạn có đoạn mã của chương trình. Bài báo đang muốn chứng minh một kỹ thuật mới để có thể thực thi dựa trên biên dịch các đoạn mã nhị phân (đồng nghĩa với việc không cần dùng đến mã nguồn chương trình). Hệ thống SymQEMU được xây dựng dựa trên QEMU, nó sẽ sửa đổi biểu diễn trung gian của chương trình mục tiêu trước khi dịch nó sang kiến trúc máy chủ. Điều này cho phép SymQEMU biên dịch các khả năng thực thi tượng trưng thành các tệp nhị phân và thu được các lợi ích về hiệu suất liên quan trong khi vẫn duy trì tính độc lập của kiến trúc.

Bài báo trình bày về hướng tiếp cận và cách thực hiện và cho thấy SymQEMU vượt trội so với các trình thực thi biểu tượng nhị phân như S2E và QSYM; trong một số tiêu chí nhất định, nó thậm chí còn đạt một hiệu suất tốt hơn cả SymCC. Ngoài ra, công cụ này còn tìm ra những lỗi hỏng chưa được phát hiện ra trong thư viện libarchive, nó chứng minh rằng sự hữu ích của công cụ trong công cuộc kiểm thử trong thực tế.

Mục lục

Phần I: Giới thiệu (Trang 3-4)
Phần II: Bối cảnh (Trang 5-10)
Phần III: SymQEMU (Trang 11-19)
Phần IV: Đánh giá (Trang 20-27)
Phần V: Kết luận (Trang 28)
Phần VI: Phụ lục (Trang 29)
Phần VII: Nguồn tham khảo (Trang 30)
Phần VIII: Phân công công việc và cam đoan (Trang 31)

I. Giới thiệu

Thực thi biểu tượng đang dần trở nên phổ biến hơn trong lĩnh vực kiểm thử phần mềm. Những nghiên cứu xuyên suốt hàng thập kỷ đã và đang cải thiện thiết kế và tăng hiệu suất của việc thực thi. Ngày nay, thực thi biểu tượng được coi là một kỹ thuật hiệu quả cao nhưng rất là tốn kém để thực thi các chương trình. Nó thường được kết hợp với kỹ thuật kiểm thử fuzz (thường được gọi là hybrid fuzzing), fuzz thường tận dụng phương pháp tìm kiếm dựa trên các thuật toán heuristic để tìm ra các đường dẫn tương đối để cải thiện tốc độ tiếp cận, trong khi thực thi tượng trưng đóng vai trò tìm ra các trường hợp thử nghiệm khó tiếp cận hơn trong chương trình mục tiêu.

Một đặc điểm quan trọng của hệ thống thực thi tượng trưng là liệu chúng có yêu cầu mã nguồn cho chương trình đang kiểm tra hay thay vào đó áp dụng cho các chương trình chỉ có nhị phân theo hướng hộp đen. Mặc dù cách kiểm thử phụ thuộc vào mã nguồn là đủ khi một người muốn kiểm thử sản phẩm của riêng họ hay là một phần mềm mã nguồn mở, có rất nhiều trường hợp thực tế yêu cầu khả năng phân tích các file nhị phân mà không cần đến sự tồn tại của mã nguồn:

- Chúng ta đang được vây quanh và phụ thuộc nhiều vào các thiết bị nhúng. Phần mềm của chúng thường có sẵn chỉ trong dạng nhị phân. Do đó, kiểm thử cũng cần các công cụ phân tích nhị phân.
- Thậm chí khi kiểm thử sản phẩm của chính mình, các phụ thuộc và thư viện không được cung cấp cùng với mã nguồn, khiến các phương pháp tiếp cận thông qua mã nguồn trở nên bất khả thi.
- Kiểm thử dựa trên mã nguồn có thể đơn giản thực hiện cho kiểm thử các chương trình lớn. Với một công cụ kiểm thử dựa trên mã nguồn, một điều thật sự cần thiết là xây dựng nên tất cả các thư viện phụ thuộc theo cách chuyên dụng do công cụ quy định, điều này có thể gây áp lực lớn cho người kiểm thử. Ngoài ra, nếu chương trình kiểm thử được xây dựng dựa trên sự kết hợp của nhiều ngôn ngữ, vấn đề xảy ra có thể là các công cụ dựa trên mã nguồn không thể thực thi tất cả mã nguồn.

Khi cần một trình thực thi biểu tượng chỉ nhị phân, người dùng thường phải đối mặt với một tình huống khó xử: có các công cụ tối ưu hóa hiệu suất hoặc tính độc lập về kiến trúc nhưng hiếm khi có công cụ nào đáp ứng được cả hai. Ví dụ, QSYM đã cho thấy hiệu suất thực thi biểu tượng rất nhanh của các tệp nhị phân, nhưng nó đạt được tốc độ cao bằng cách liên kết việc thực thi với bộ lệnh của bộ xử lý x86. Điều này không chỉ khiến hệ thống phụ thuộc vào kiến trúc mà còn làm tăng tính phức tạp của hệ thống do kích thước quá lớn của bộ lệnh của bộ xử lý hiện đại; theo lời của chính tác giả, cách tiếp cận của họ là "Chấp nhận sự phức tạp và khó khăn trong việc triển khai để giảm thiểu chi phí". Ngược lại, S2E là một ví dụ về một hệ thống có thể áp dụng rộng rãi, linh hoạt nhưng lại có tốc độ thực thi tương đối thấp. Về mặt khái niệm, S2E có thể phân tích mã cho hầu hết các kiến trúc CPU, bao gồm cả mã hạt nhân. Tuy nhiên, tính ứng dụng rộng rãi của nó đạt được nhờ qua nhiều bước chuyển đổi và cuối cùng là diễn giải chương trình mục tiêu (sẽ được trình bày chi tiết sau), điều này làm tăng độ phức tạp của hệ thống và cuối cùng ảnh hưởng đến hiệu suất. Thực tế, hiệu suất cao trong phân tích biểu tượng chỉ dựa trên nhị phân thường đạt được nhờ các triển khai chuyên biệt hóa cao — một lựa chọn thiết kế mâu thuẫn với tính linh hoạt của kiến trúc.

Trong bài báo này, tác giả trình bày một phương pháp thay thế mà không phụ thuộc vào kiến trúc mục tiêu của chương trình được kiểm tra, có độ phức tạp triển khai thấp, nhưng đạt được hiệu suất cao. Ý tưởng chính của hệ thống SymQEMU, là kết hợp cơ chế giả lập CPU của QEMU với một cơ chế thực thi biểu tượng rất nhẹ: thay vì thực hiện quá trình chuyển đổi tốn kém về mặt tính toán của chương trình mục tiêu sang một biểu diễn trung gian và sau đó giải thích biểu tượng (như trong S2E), SymQEMU tận dụng cơ chế dịch nhị phân của QEMU để biên dịch xử lý biểu tượng trực tiếp vào mã máy mà trình giả lập phát sinh và thực thi. Cách tiếp cận này mang lại hiệu suất vượt trội so với các hệ thống tiên tiến nhất hiện nay trong khi vẫn giữ được tính độc lập hoàn toàn với nền tảng. Hiện tại, bài báo tập trung vào các chương trình ở chế độ người dùng trên Linux (tức là các tệp nhị phân ELF), nhưng hoàn toàn có thể mở rộng khái niệm này để giả lập toàn hệ thống cho các nền tảng bất kỳ mà QEMU hỗ trợ (ví dụ như để phân tích firmware). Hơn nữa, SymQEMU được công khai để thúc đẩy các nghiên cứu trong lĩnh vực này trong tương lai.

Lưu ý rằng biên dịch biểu tượng thực thi xử lý vào các chương trình đích là cốt lõi chính của công trình nghiên cứu của nhóm tác giả với SymCC. Nhóm tác giả đã chứng minh rằng SymCC vượt trội hơn các phương pháp thực thi biểu tượng hiện tại khác. Tuy nhiên, SymCC chỉ áp dụng được khi mã nguồn có sẵn, do đó không hỗ trợ phân tích nhị phân. Ngược lại, SymQEMU cho thấy cách đạt được hiệu suất tương tự trong môi trường chỉ có nhị phân, đồng thời tuân thủ tất cả các ràng buộc bổ sung vốn có trong bối cảnh này. Để biết thêm chi tiết về so sánh giữa SymQEMU và SymCC, chúng ta sẽ cùng phân tích sau

Bài báo đã so sánh SymQEMU với các trình thực thi biểu tượng nhị phân tiên tiến hiện nay như S2E và QSYM, và nhận thấy rằng SymQEMU vượt trội hơn cả hai về phạm vi bao phủ đạt được theo thời gian. Hơn nữa, bài báo cho thấy hiệu suất của SymQEMU tương đương với SymCC, mặc dù SymCC yêu cầu quyền truy cập vào mã nguồn. Cuối cùng, bài báo đã gửi SymQEMU đến Google FuzzBench, một khung so sánh dành cho các công cụ fuzzing; mặc dù bộ kiểm tra không hướng đến các hệ thống thực thi biểu tượng, SymQEMU đã vượt trội hơn tất cả các công cụ fuzzing được đưa vào thử nghiệm ở 3 trên 21 tiêu chí

Tóm lại, nhóm tác giả đã làm được những đóng góp sau đây:

- Phân tích các triển khai tiên bộ của thực thi biểu tượng chỉ dựa trên nhị phân và xác định các điểm mạnh cũng như điểm yếu tương ứng trong thiết kế của chúng.
- Trình bày cách tiếp cận kết hợp ưu điểm của các hệ thống đã có trước đây trong khi đó tránh đi hầu hết những nhược điểm của chúng; ý tưởng chính là một kỹ thuật mới để áp dụng thực thi biểu tượng dựa trên biên dịch cho các tệp nhị phân. Mã nguồn của hệ thống được cung cấp là một mã nguồn mở.
- Đánh giá SymQEMU trên Google FuzzBench, cũng như trên phần mềm thực tế mã nguồn mở và mã nguồn đóng. Kết quả thô, cùng với dữ liệu và các tập lệnh được sử dụng trong quá trình đánh giá, sẽ được công bố cùng với bài báo

Phần còn lại của bài báo được cấu trúc như sau: Đầu tiên, tổng quan về thực thi biểu tượng, đặc biệt là dạng chỉ dành cho nhị phân, và các triển khai hiện có của nó. Sau đó, trình bày thiết kế và triển khai SymQEMU và đánh giá nó so với các công nghệ hiện đại hiện nay. Cuối cùng, thảo luận về công việc trong tương lai, đặt đóng góp của nhóm tác giả trong bối cảnh các nghiên cứu trước đây, và đưa ra kết luận

II. Bối cảnh

Trong phần này, bài báo giới thiệu thực thi biểu tượng một cách tổng quan trước khi xem xét các thách thức của thực thi biểu tượng chỉ dành cho nhị phân và đánh giá cách các triển khai hiện đại giải quyết chúng. Cuối cùng, thảo luận về SymCC, một trình thực thi biểu tượng dựa trên mã nguồn, đóng vai trò là nguồn cảm hứng quan trọng cho công trình của nhóm tác giả.

A. Thực thi biểu tượng

Mục tiêu chung của thực thi biểu tượng là theo dõi cách các giá trị trung gian được tính toán trong quá trình thực thi của một chương trình mục tiêu. Thông thường, mỗi giá trị trung gian có thể được biểu diễn dưới dạng một công thức liên quan đến các đầu vào của chương trình. Sau đó, tại bất kỳ thời điểm nào trong quá trình thực thi, hệ thống có thể tận dụng các biểu thức biểu tượng này để trả lời các câu hỏi như: "Liệu truy cập mảng này có thể vượt giới hạn không?", "Có thể đi theo nhánh này của chương trình không?" hoặc "Con trỏ này có thể bằng null khi được giải tham chiếu không?"

Hơn nữa, nếu câu trả lời là khẳng định, các trình thực thi biểu tượng thường cung cấp một trường hợp kiểm thử, tức là một đầu vào chương trình mới kích hoạt hành vi được yêu cầu. Khả năng này làm cho thực thi biểu tượng cực kỳ hữu ích cho việc kiểm thử chương trình tự động, nơi mục tiêu là khám phá càng nhiều trường hợp biên của chương trình càng tốt và tìm ra các đầu vào gây ra lỗi hoặc kích hoạt các lỗi khác.

Để theo dõi các phép tính trong chương trình mục tiêu, các hệ thống thực thi biểu tượng cần phải có một mức độ hiểu biết nhất định về tập lệnh của chương trình. Nhiều triển khai hiện tại chuyển đổi chương trình sang một biểu diễn trung gian; các ví dụ điển hình về biểu diễn trung gian bao gồm LLVM bitcode và VEX. Biểu diễn trung gian sau đó được thực thi theo cách biểu tượng; vì trình thực thi chỉ cần xử lý ngôn ngữ trung gian (thường bao gồm một số lượng lệnh tương đối ít), nên việc triển khai có thể đơn giản hơn đáng kể. Hơn nữa, trong các nghiên cứu trước đây, các truy vấn được dẫn xuất từ các biểu diễn cấp cao của chương trình cần kiểm thử để giải quyết hơn so với các truy vấn được dẫn xuất từ các tập lệnh cấp thấp như mã máy.

Tuy nhiên, việc chuyển đổi chương trình sang các biểu diễn trung gian đòi hỏi nỗ lực tính toán và tạo ra chi phí bổ sung trong quá trình thực thi chương trình; do đó, một số triển khai chọn cách bỏ qua bất kỳ hình thức chuyển đổi nào và làm việc trực tiếp trên mã máy. Ngoài những lợi ích về hiệu năng, việc bỏ qua bước chuyển đổi chương trình còn giúp cải thiện tính bền vững, vì mã máy cụ thể có thể được thực thi ngay cả khi trình thực thi biểu tượng không biết cách diễn giải một lệnh cụ thể. Tuy nhiên, nhược điểm của cách tiếp cận này là việc chuyên biệt hóa trên mã máy của một kiến trúc bộ xử lý cụ thể sẽ giới hạn hệ thống thực thi biểu tượng trong nền tảng đó. Ở thái cực ngược lại, làm việc trực tiếp trên mã nguồn ngày nay ít phổ biến hơn và rõ ràng không áp dụng được trong trường hợp chỉ có tệp nhị phân.

B. Thực thi biểu tượng chỉ dành cho nhị phân

Yêu cầu hệ thống phân tích làm việc chỉ với tệp nhị phân mang lại một tập hợp các thách thức riêng biệt: Khi không có mã nguồn, việc chuyển đổi chương trình sang biểu diễn trung gian đòi hỏi các trình dịch ngược đáng tin cậy. Do những khó khăn của việc dịch ngược tĩnh, hầu hết các triển khai thực hiện quá trình chuyển đổi khi cần thiết trong thời gian chạy. Hơn nữa, việc hỗ trợ nhiều kiến trúc trở nên quan trọng khi mã nguồn không có sẵn: nếu không có mã nguồn, việc biên dịch chéo một chương trình cho bất kỳ kiến trúc nào mà trình thực thi biểu tượng hỗ trợ là không khả thi. Nếu một hệ thống thực thi biểu tượng không thể xử lý kiến trúc mục tiêu của chương trình cần kiểm thử, nó đơn giản là không thể được sử dụng. Điều này đặc biệt quan trọng trong lĩnh vực nhúng, nơi mà sự đa dạng lớn về kiến trúc bộ xử lý là điều phổ biến.

Các trình thực thi biểu tượng không sử dụng chuyển đổi do đó phải đối mặt với thách thức về khả năng di động trong kịch bản chỉ dành cho nhị phân, cùng với các vấn đề về khả năng bảo trì phát sinh từ việc triển khai tương đối phức tạp. Trong khi đó, các trình thực thi chuyển đổi chương trình mục tiêu sang biểu diễn trung gian đạt được kết quả tốt hơn, nhưng vẫn yêu cầu một trình chuyển đổi đáng tin cậy cho kiến trúc mục tiêu cụ thể; một lượng lớn công việc đã được đầu tư để xác minh tính đúng đắn của các trình chuyển đổi. Điều này trái ngược với thực thi biểu tượng dựa trên mã nguồn, nơi mà các biểu diễn trung gian có thể được tạo ra khá dễ dàng từ mã nguồn của chương trình.

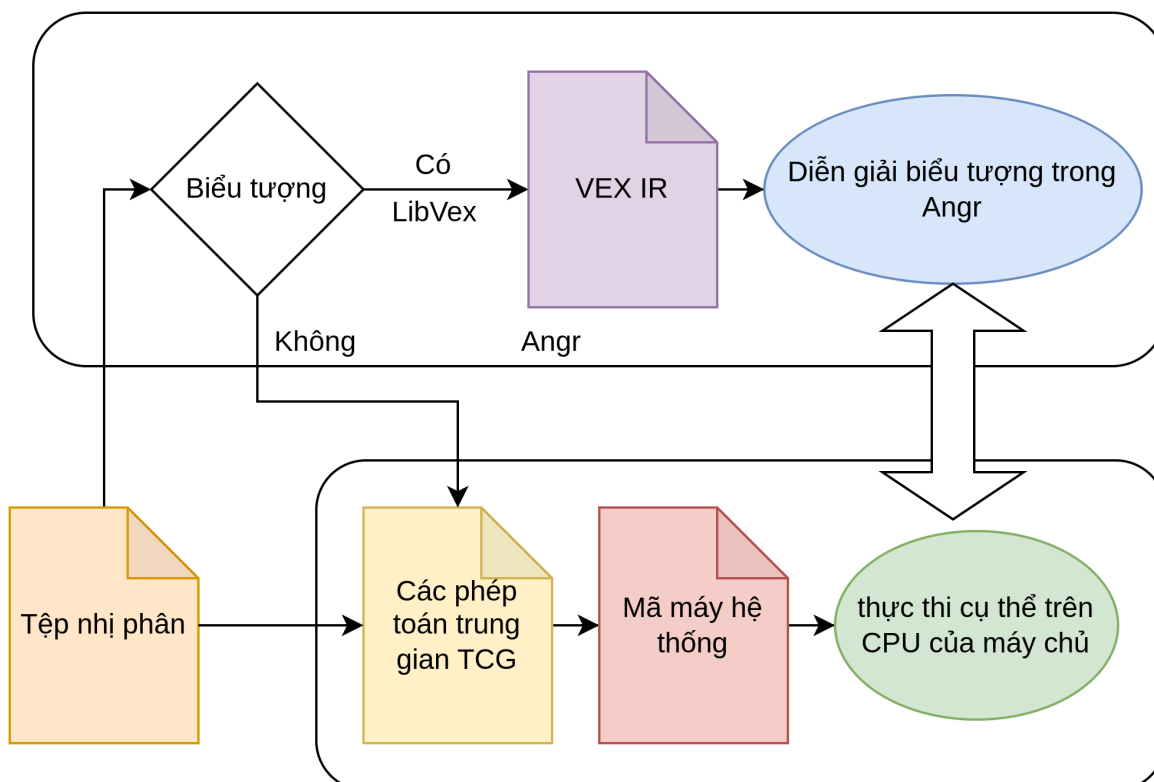
Tóm lại, thực thi ký hiệu chỉ dành cho nhị phân đặt ra yêu cầu cao hơn về tính linh hoạt trong kiến trúc và hiệu suất của quá trình chuyển đổi chương trình (thời gian chạy) so với phân tích dựa trên mã nguồn.

C. Các giải pháp hiện đại

Sau khi trình bày những thách thức của thực thi ký hiệu chỉ dành cho nhị phân, tác giả mô tả ba triển khai tiên tiến phổ biến và nghiên cứu các lựa chọn thiết kế mà chúng sử dụng để giải quyết những vấn đề này.

1) Angr: Trình thực thi ký hiệu "kính hiển vi" dựa trên chuyển đổi

Angr tái sử dụng VEX, ngôn ngữ trung gian và trình chuyển đổi của khung công cụ Valgrind. Các chương trình mục tiêu được chuyển đổi tại thời gian chạy; trình thực thi ký hiệu sau đó diễn giải các lệnh VEX. Như một tối ưu hóa, Angr có thể thực thi các phép tính không liên quan đến dữ liệu ký hiệu (tức là các phép tính mà kết quả không phụ thuộc vào đầu vào chương trình) trong Unicorn, một trình giả lập CPU nhanh dựa trên QEMU. (Hình 1 minh họa thiết kế này.)



Hình 1: Tổng quan về angr: Chương trình mục tiêu được nâng lên ngôn ngữ trung gian VEX IR và sau đó được diễn giải theo ký hiệu hoặc thực thi một cách cụ thể trong trình giả lập CPU Unicorn

Do dựa trên VEX, Angr thừa hưởng khả năng hỗ trợ tất cả các kiến trúc mà VEX có thể xử lý. Tuy nhiên, vì lõi của trình thực thi ký hiệu được viết bằng Python, Angr khá chậm nhưng lại rất linh hoạt.

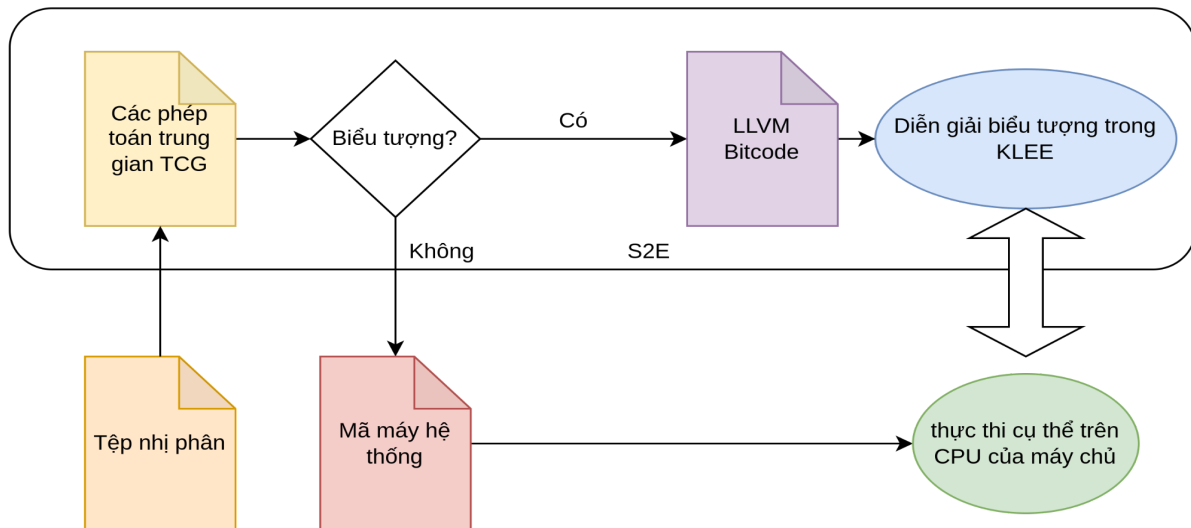
2) S2E:

Được tạo ra từ mong muốn mở rộng phạm vi của hệ thống thực thi biểu tượng dựa trên mã nguồn KLEE đến các phụ thuộc của chương trình mục tiêu và hệ điều hành. Để đạt được mục tiêu này, S2E chạy toàn bộ hệ điều hành trong trình giả lập QEMU và kết nối nó với KLEE để thực thi mã liên quan theo cách biểu tượng (xem Hình 2). Hệ thống kết quả khá phức tạp, bao gồm nhiều lần chuyển đổi chương trình cần kiểm thử:

- QEMU là một trình dịch mã nhị phân, tức là trong quá trình hoạt động bình thường, nó chuyển đổi chương trình mục tiêu từ mã máy sang một biểu diễn trung gian (gọi là TCG ops), sau đó biên dịch lại thành mã máy cho CPU của máy chủ.
- Khi các phép toán liên quan đến dữ liệu ký hiệu, QEMU đã được sửa đổi mà S2E sử dụng không biên dịch lại các TCG ops thành mã máy của máy chủ; thay vào đó, nó chuyển đổi chúng thành LLVM bytecode, sau đó được chuyển đến KLEE.
- KLEE sẽ diễn giải LLVM bytecode theo cách ký hiệu và trả lại phần cụ thể của kết quả cho QEMU.

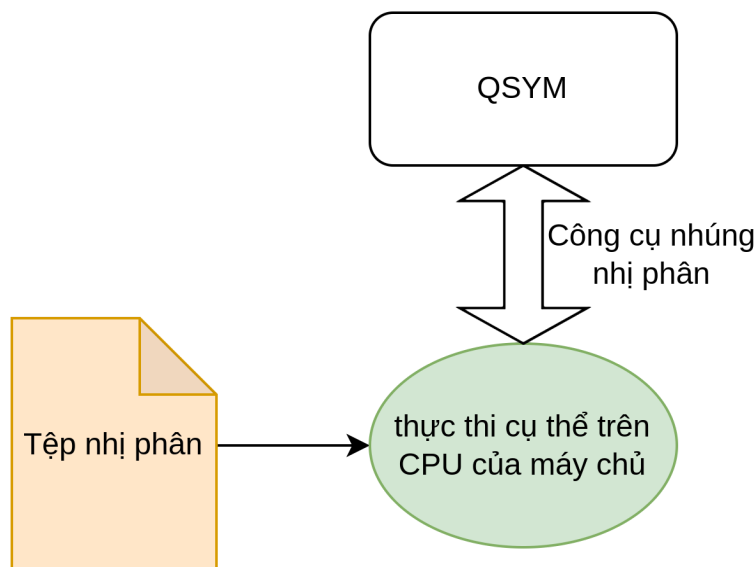
Cách tiếp cận này tạo ra một hệ thống rất linh hoạt, có thể xử lý về lý thuyết nhiều kiến trúc khác nhau và theo dõi các phép toán qua tất cả các lớp của hệ điều hành. Tuy

nhân, sự linh hoạt này đi kèm với một cái giá: S2E là một hệ thống phức tạp với một mã nguồn lớn. Hơn nữa, việc chuyển đổi hai bước từ mã máy sang TCG ops và từ đó đến LLVM bytecode làm giảm hiệu suất của nó. So với angr từ góc độ người dùng, S2E phức tạp hơn để thiết lập và chạy, nhưng cung cấp một phân tích toàn diện hơn.



Hình 2. Tổng quan về S2E: chương trình mục tiêu được nâng lên TCG ops và sau đó hoặc được chuyển đổi thành mã máy của máy chủ, hoặc lại được nâng thêm một bước nữa và thực thi ký hiệu trong KLEE.

3) QSYM:



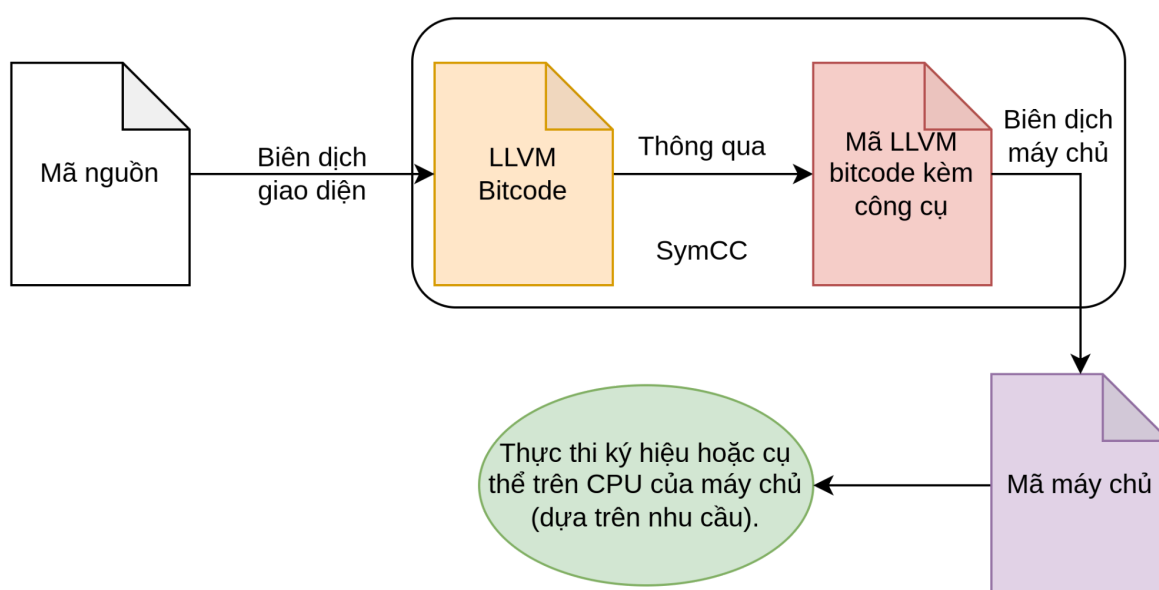
Hình 3. Tổng quan về QSYM: chương trình mục tiêu được thực thi trực tiếp trên CPU trong khi QSYM nhúng mã vào nó một cách động.

Với cốt lõi là tập trung vào hiệu suất, QSYM không chuyển đổi chương trình mục tiêu sang ngôn ngữ trung gian. Thay vào đó, nó thêm mã công cụ vào mã máy x86 trong thời gian chạy để thêm theo dõi biểu tượng vào các tệp nhị phân (xem Hình 3). Cụ thể, nó sử dụng Intel Pin, một khung công cụ nhị phân động, để chèn các điểm móc vào chương trình mục tiêu. Bên trong các điểm móc này, nó thực hiện tương đương ký hiệu của các lệnh mã máy mà chương trình thực thi.

Thiết kế này mang lại một trình thực thi ký hiệu rất nhanh và mạnh mẽ cho các chương trình x86. Tuy nhiên, hệ thống này về bản chất bị giới hạn chỉ với một kiến trúc mục tiêu duy nhất, và việc triển khai khá tẻ nhạt vì nó cần xử lý từng lệnh x86 mà có thể xảy ra trong các phép toán liên quan. Trong các công trình trước đây, nhóm tác giả đã thấy QSYM là một công cụ tuyệt vời để phân tích các tệp nhị phân x86, nhưng việc thêm hỗ trợ cho một kiến trúc khác sẽ là một lượng công việc đáng kể.

D. SymCC:

Trình thực thi ký hiệu SymCC, được giới thiệu gần đây bởi cùng tác giả với bài báo hiện tại, không hoạt động trên các tệp nhị phân; tuy nhiên, SymQEMU đã lấy cảm hứng từ SymCC, vì vậy tác giả sẽ tóm tắt thiết kế của SymCC ở đây. tác giả giới thiệu cho độc giả quan tâm về bài báo gốc để biết chi tiết.



Hình 4. Tổng quan về SymCC: Mã nguồn của chương trình mục tiêu được biên dịch thành mã máy; xử lý ký hiệu được chèn vào ở mức LLVM bitcode trong trình biên dịch.

Quan sát chính của tác giả khi xây dựng SymCC là hầu hết các hệ thống thực thi ký hiệu hiện đại đều là các trình thông dịch. Thay vào đó, tác giả đề xuất một phương pháp dựa trên biên dịch, chỉ ra rằng phương pháp này cải thiện hiệu suất thực thi cũng như khả năng khám phá tổng thể của hệ thống kết quả. SymCC can thiệp vào các trình biên dịch và nhúng mã công cụ vào mã mục tiêu tại thời điểm biên dịch, chèn các lệnh gọi đến thư viện hỗ trợ

thời gian chạy. Do đó, thực thi ký hiệu trở thành một phần không thể tách rời của chương trình đã biên dịch. Hơn nữa, mã phân tích hưởng lợi từ các tối ưu hóa của trình biên dịch, và công việc nhúng mã không bị trùng lặp trong mỗi lần thực thi. (Hình 4 minh họa thiết kế này.)

Phương pháp dựa trên biên dịch của SymCC yêu cầu một trình biên dịch, vì vậy nó chỉ có thể áp dụng khi mã nguồn của chương trình cần kiểm thử có sẵn. Tuy nhiên, tác giả cho rằng phương pháp này đủ hứa hẹn để tìm cách áp dụng nó vào thực thi ký hiệu trên các tệp nhị phân. Một đóng góp quan trọng của bài báo này là chứng minh cách thực thi ký hiệu dựa trên biên dịch có thể, thực sự, được làm việc hiệu quả trên các tệp nhị phân.

III. SYM QEMU

Bài báo trình bày thiết kế và triển khai của trình thực thi ký hiệu chỉ dành cho tệp nhị phân SymQEMU. Nó kế thừa từ các nghiên cứu trước đây và kết hợp những ưu điểm của các hệ thống hiện đại với các ý tưởng mới để tạo ra một công cụ phân tích vừa nhanh vừa linh hoạt.

A. Thiết kế:

Hệ thống có hai mục tiêu chính:

1. Đạt được hiệu suất cao để có thể mở rộng phân tích đến các phần mềm trong thế giới thực.
2. Duy trì tính độc lập nền tảng ở mức hợp lý, tức là việc thêm hỗ trợ cho một kiến trúc vi xử lý không nên đòi hỏi quá nhiều công sức.

Dựa trên việc khảo sát về các giải pháp hiện đại ở Mục II-C, bài báo cho rằng các hệ thống tiên tiến phổ biến thường chỉ đạt được một trong hai mục tiêu, nhưng không cả hai: trong số các hệ thống được trình bày, S2E và angr có tính linh hoạt cao nhưng lại kém hiệu suất hơn, trong khi QSYM rất nhanh nhưng lại gắn chặt với nền tảng x86.

Bài báo cho rằng các giải pháp hiện tại, vốn độc lập với nền tảng, thường chuyển chương trình cần kiểm thử sang một dạng biểu diễn trung gian. Theo cách này, để hỗ trợ một kiến trúc mới, chỉ cần chuyển đổi phần dịch của trình biên dịch. Lý tưởng nhất, người ta chọn một ngôn ngữ trung gian đã có các bộ dịch cho nhiều kiến trúc liên quan. Việc biểu diễn chương trình theo cách không phụ thuộc vào kiến trúc để đảm bảo tính linh hoạt là một kỹ thuật đã được biết đến rộng rãi và được áp dụng thành công trong nhiều lĩnh vực khác, như thiết kế trình biên dịch và phân tích nhị phân phân tĩnh. Do đó, tác giả cũng tích hợp nó vào thiết kế của mình.

Mặc dù việc chuyển đổi chương trình sang dạng biểu diễn trung gian mang lại tính linh hoạt, tác giả cần lưu ý đến tác động đến hiệu suất: chuyển đổi các chương trình chỉ ở dạng nhị phân theo cách tĩnh là một thách thức vì việc dịch ngược mã có thể không đáng tin cậy (đặc biệt khi có các lệnh nhảy gián tiếp), và thực hiện chuyển đổi trong thời gian chạy dẫn đến chi phí bổ sung trong quá trình phân tích.

Đây là lý do cốt lõi tại sao các hệ thống thực thi biểu tượng có sự chuyển đổi linh hoạt như S2E và angr tụt hậu về hiệu suất so với các hệ thống không dịch chuyển như QSYM. Mục tiêu của tác giả là tìm ra cách xây dựng một hệ thống có sự chuyển đổi linh hoạt mà vẫn đạt hiệu suất cao.

Trước hết, tốc độ của cả S2E và angr bị ảnh hưởng bởi các vấn đề không cốt lõi, có thể được khắc phục bằng nỗ lực kỹ thuật:

- S2E dịch chương trình cần kiểm thử hai lần (xem Mục II-C2). Việc dịch lần thứ hai có thể được loại bỏ nếu thực thi biểu tượng được triển khai ngay trên dạng biểu diễn trung gian đầu tiên.

- Hiệu suất của angr bị ảnh hưởng do được triển khai bằng Python; việc chuyển đổi phần lõi sang một ngôn ngữ lập trình nhanh hơn có khả năng mang lại cải thiện đáng kể về tốc độ.

Tuy nhiên, bài báo không chỉ dừng lại ở việc xác định và tránh hai vấn đề trên. Tại đây, một quan sát thứ hai được đưa ra: Cả S2E và angr, cũng như tất cả các trình thực thi ký hiệu chỉ dành cho nhị phân có dịch chuyển mà tác giả đề cập, đều thực thi dưới dạng diễn dịch (interpret) dạng biểu diễn trung gian của chương trình cần kiểm thử. (Điều này không phụ thuộc vào các sửa đổi được đề xuất ở trên-diễn dịch là một phần cốt lõi trong thiết kế của chúng.)

Tác giả đưa ra giả thuyết rằng việc biên dịch một phiên bản đã được gắn công cụ của chương trình mục tiêu sẽ mang lại hiệu suất cao hơn rất nhiều. SymCC gần đây đã chỉ ra rằng điều này đúng đối với thực thi ký hiệu dựa trên mã nguồn, nhưng thiết kế dựa trên trình biên dịch của nó vốn dĩ yêu cầu mã nguồn và do đó không áp dụng cho trường hợp chỉ có tệp nhị phân (xem Mục II-D).

Cách tiếp cận của bài báo, được lấy cảm hứng từ các quan sát trên, như sau:

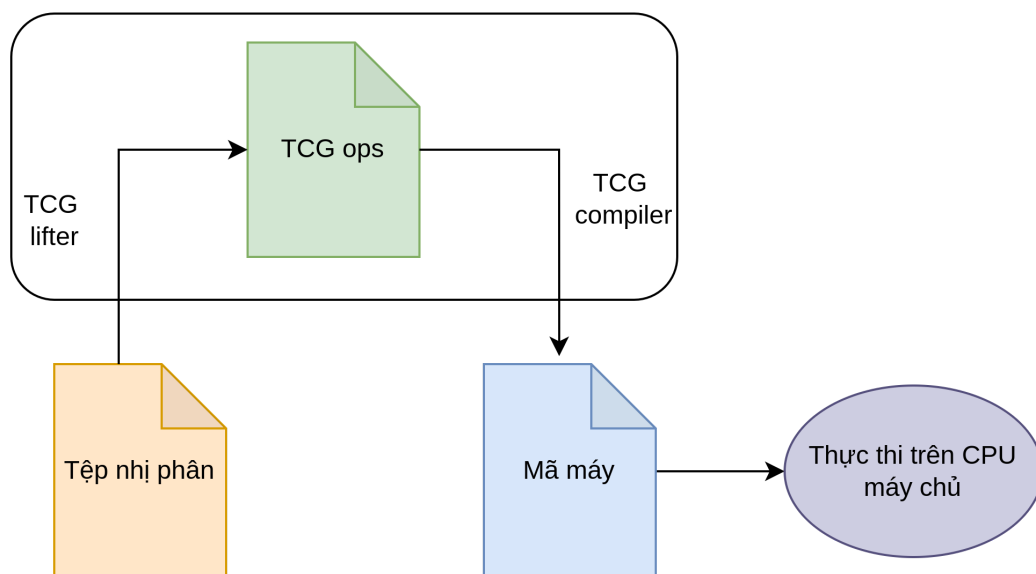
1. Dịch chương trình mục tiêu sang một ngôn ngữ trung gian trong thời gian chạy.
2. Gắn công cụ vào dạng biểu diễn trung gian khi cần thiết để thực thi ký hiệu.
3. Biên dịch dạng biểu diễn trung gian thành mã máy phù hợp với CPU đang thực thi phân tích và chạy trực tiếp.

Bằng cách biên dịch chương trình mục tiêu đã được gắn công cụ thành mã máy, tác giả bù đắp được chi phí hiệu suất do việc dịch tệp nhị phân sang ngôn ngữ trung gian ban đầu gây ra: CPU thực thi mã máy nhanh hơn nhiều so với việc trình diễn dịch chạy dạng biểu diễn trung gian, nhờ đó đạt được hiệu suất tương đương với một hệ thống không dịch chuyển, đồng thời vẫn giữ được ưu điểm về tính độc lập kiến trúc mà dịch chuyển chương trình mang lại.

B. Thực hiện:

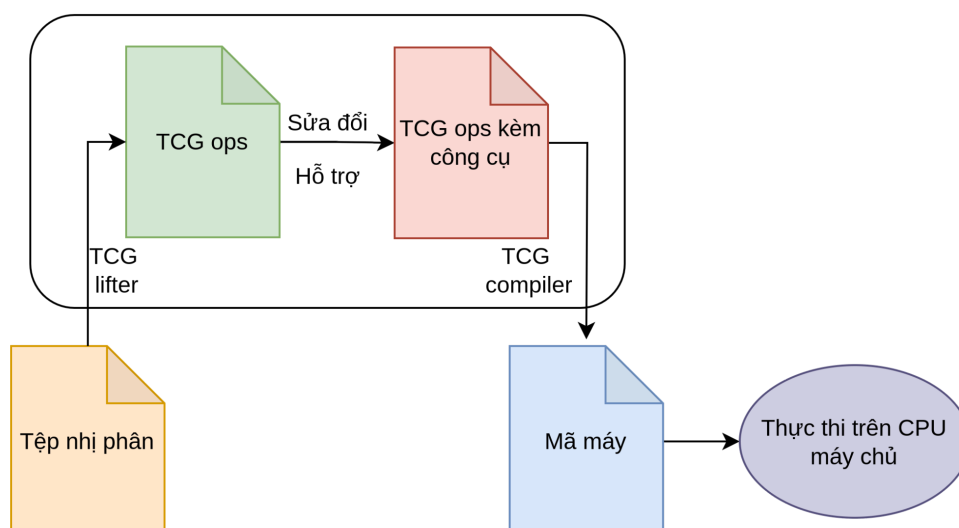
Tác giả đã triển khai SymQEMU dựa trên QEMU, như được gợi ý bởi tên gọi. Lí do tác giả chọn QEMU vì đây là một trình giả lập hệ thống mạnh mẽ, hỗ trợ rất nhiều kiến trúc. Bằng cách xây dựng trên nền tảng này để đạt được mục tiêu về tính độc lập nền tảng. Cần lưu ý rằng S2E cũng được xây dựng dựa trên QEMU, có lẽ vì lý do tương tự. Nhưng QEMU còn có một đặc điểm khác đáp ứng nhu cầu và phân biệt nó với các trình dịch khác: QEMU không chỉ dịch các tệp nhị phân sang dạng biểu diễn trung gian không phụ thuộc vào kiến trúc bộ xử lý, mà còn có khả năng biên dịch ngôn ngữ trung gian này thành mã máy cho CPU máy chủ. tác giả tận dụng cơ chế này để đạt được mục tiêu thứ hai: hiệu suất.

Lưu ý: framework Valgrind cũng hỗ trợ một cơ chế tương tự nhưng ưu điểm chính của QEMU so với Valgrind trong trường hợp của tác giả là QEMU có thể dịch tệp nhị phân từ một kiến trúc khách cụ thể sang mã máy cho một kiến trúc máy chủ khác, cũng như giả lập toàn bộ hệ thống, điều này làm cho nó trở thành cơ sở tốt hơn để mở rộng hỗ trợ phân tích firmware đa kiến trúc trong tương lai.



Hình 5. **Tổng quan về QEMU thông thường:** Chương trình mục tiêu được dịch sang **TCG ops**, sau đó được biên dịch thành mã máy và thực thi trên CPU máy chủ.

Cụ thể, bài báo cho biết mở rộng một thành phần trong QEMU được gọi là Tiny Code Generator (TCG). Trong QEMU chưa sửa đổi, TCG chịu trách nhiệm dịch các khối mã máy của kiến trúc khách thành một ngôn ngữ không phụ thuộc kiến trúc gọi là TCG ops, sau đó biên dịch các TCG ops này thành mã máy cho kiến trúc máy chủ (xem Hình 5). Các khối được dịch sau đó được lưu vào bộ đệm để tăng hiệu suất, do đó quá trình dịch chỉ cần thực hiện một lần trong mỗi lần chạy. SymQEMU chèn thêm một bước vào quy trình này: Trong khi chương trình cần kiểm thử được dịch sang TCG ops, tác giả không chỉ tạo ra các lệnh để mô phỏng CPU khách mà còn bổ sung thêm các TCG ops để xây dựng các biểu thức ký hiệu cho các kết quả (xem Hình 6).



Hình 6. **Tổng quan về SymQEMU:** Chương trình mục tiêu được dịch sang TCG ops giống như trong QEMU thông thường (xem Hình 5), nhưng trước khi biên dịch thành mã máy cho CPU máy chủ, chúng ta chèn thêm các lệnh để thực hiện thực thi biểu tượng trong quá trình chạy.

Ví dụ, giả sử rằng một hàm trong chương trình mục tiêu cộng thêm hằng số 42 vào một số nguyên đầu vào (sử dụng mã C cho ví dụ này):

```
int add42(int x) {  
  
    return x + 42;  
  
}
```

Với tối ưu hóa được bật, GCC sẽ nội suy (inline) hàm và chuyển nó thành lệnh assembly này khi biên dịch cho kiến trúc x86-64:

```
lea esi,[rax+0x2a]
```

Mã máy là tất cả những gì SymQEMU nhận được; nó không có quyền truy cập vào mã nguồn. Khi thực thi chương trình mục tiêu, TCG tạo ra đại diện độc lập với kiến trúc của mã máy sau:

```
movi_i64 tmp12, $0x2a  
  
add_i64 tmp2, rax, tmp12  
  
ext32u_i64 rsi, tmp2
```

Lưu ý rằng các đối số của TCG ops được sắp xếp giống như cú pháp assembly x86 theo kiểu Intel, tức là đích đến là đối số đầu tiên của bất kỳ lệnh nào. Các lệnh trên thực hiện phép cộng 64-bit và lưu kết quả dưới dạng một số nguyên 32-bit. QEMU thông thường sẽ dịch các TCG ops này thành mã máy cho kiến trúc máy chủ. Tuy nhiên, SymQEMU sẽ chèn các lệnh bổ sung để tính toán ký hiệu trước khi mã được dịch sang kiến trúc máy chủ:

```
movi_i64 tmp12_expr, $0x0  
  
movi_i64 tmp12, $0x2a  
  
call sym_add_i64, $0x5, $1, tmp2_expr, rax, rax_expr, tmp12,  
tmp12_expr  
  
add_i64 tmp2, rax, tmp12  
  
movi_i64 tmp12, $0x4  
  
call sym_zext, $0x5, $1, rsi_expr, tmp2_expr, tmp12  
  
ext32u_i64 rsi, tmp2
```

Mỗi khối mã tương ứng với một trong các TCG ops do QEMU tạo ra ban đầu; thực tế, lệnh cuối cùng của mỗi khối là giống hệt với lệnh gốc tương ứng. Trong khối đầu tiên, chúng ta đặt biểu thức liên quan đến hằng số 42 thành null (tức là chúng ta khai báo giá trị này là cụ thể). Trong khối thứ hai, hàm hỗ trợ sym_add_i64 tạo ra một biểu thức ký hiệu đại

diện cho phép cộng của hai số nguyên 64-bit (sử dụng `rax_expr`, biểu thức tương ứng với đầu vào của hàm). Cuối cùng, khối cuối cùng gọi hàm hỗ trợ `sym_zext` với đối số 4 để xây dựng một biểu thức chuyển đổi kết quả của phép cộng thành một giá trị 4 byte (tức là 32-bit). Quan trọng là, SymQEMU không thực hiện bất kỳ cuộc gọi nào đến thư viện hỗ trợ trong thời gian dịch (như một trình thông dịch sẽ làm)—nó chỉ phát ra các TCG ops tương ứng và dựa vào các cơ chế QEMU thông thường để dịch chúng thành mã máy. Cách này, các công thức ký hiệu được xây dựng trong mã máy gốc mà không phải chịu chi phí liên quan đến việc thông dịch một ngôn ngữ trung gian.

Đối với thư viện hỗ trợ xây dựng các biểu thức ký hiệu và giải quyết các truy vấn trên chúng, tác giả tái sử dụng mã từ SymCC, vốn lại dựa trên QSYM. Lợi thế của cách này, ngoài việc giúp tác giả không phải tái triển khai những gì đã hoạt động tốt trong QSYM, là nó loại bỏ một nguồn gây nhiễu từ việc đánh giá của tác giả: vì SymQEMU và QSYM sử dụng cùng một logic để xây dựng và đơn giản hóa các biểu thức, cũng như tương tác với bộ giải, có thể chắc chắn rằng những sự khác biệt về hiệu suất quan sát được không đến từ các khía cạnh thiết kế ngoài lề đó.

Tác giả hiện tại sử dụng mô phỏng chế độ người dùng của QEMU, tức là chỉ mô phỏng không gian người dùng của hệ thống khách. Các cuộc gọi hệ thống được dịch để đáp ứng yêu cầu của kiến trúc máy chủ, và chúng được thực thi chống lại kernel của máy chủ (sử dụng các cơ chế thông thường của QEMU). Do đó, phân tích ký hiệu của tác giả dừng lại tại ranh giới cuộc gọi hệ thống, tương tự như QSYM và angr. So với mô phỏng toàn bộ hệ thống (như S2E thực hiện), cách làm này tiết kiệm công sức chuẩn bị các hình ảnh hệ điều hành cho mỗi kiến trúc mục tiêu và tăng hiệu suất bằng cách chạy mã kernel một cách cụ thể và không cần mô phỏng. Tuy nhiên, lưu ý rằng SymQEMU có thể được mở rộng để làm việc với mô phỏng toàn bộ hệ thống của QEMU nếu cần thiết (xem Mục V).

Tổng thể, SymQEMU thêm khoảng 2,000 dòng mã C vào QEMU. Hơn nữa, tác giả đã thêm khoảng 100 dòng C++ vào thư viện hỗ trợ của SymCC để hỗ trợ phương pháp quản lý bộ nhớ của tác giả.

C. Tính độc lập với nền tảng:

Hỗ trợ nhiều kiến trúc CPU là một mục tiêu quan trọng của SymQEMU ngay từ đầu. Vì vậy, tác giả sẽ xem xét chi tiết mức độ mà hệ thống của tác giả đạt được mục tiêu này.

Trước tiên, điều quan trọng là phải phân biệt giữa kiến trúc của máy tính chạy phân tích (thường được gọi là máy chủ) và kiến trúc mà chương trình thử nghiệm được biên dịch cho (máy khách trong thuật ngữ của QEMU). Đặc biệt trong phân tích phần mềm nhúng, việc máy chủ và máy khách có kiến trúc khác nhau là điều mong muốn—thiết bị nhúng mà phần mềm nhúng đang thử nghiệm có thể thiếu khả năng tính toán để thực hiện phân tích biểu tượng với tốc độ hợp lý, vì vậy người ta thường chạy công cụ thực thi biểu tượng (và nói chung, bất kỳ bài kiểm tra phần mềm nhúng nào) trên một máy tính mạnh mẽ hơn. SymQEMU đã chuẩn bị tốt cho trường hợp sử dụng này: QEMU có thể chạy trên tất cả các kiến trúc máy chủ chính.

Vậy còn các kiến trúc máy khách thì sao? SymQEMU tận dụng bộ dịch TCG của QEMU, hỗ trợ một loạt các loại bộ xử lý—tài liệu trực tuyến hiện tại liệt kê 22 nền tảng, bao

gồm x86, ARM, MIPS và Xtensa, mỗi nền tảng bao gồm nhiều loại bộ xử lý. Hơn nữa, các sửa đổi của tác giả gần như hoàn toàn độc lập với nền tảng mục tiêu: trong số 2.000 dòng mã C mà tác giả đã thêm vào QEMU, chỉ có 10 dòng là đặc thù cho kiến trúc máy khách (tức là x86 trong các thí nghiệm của tác giả). Cụ thể, chúng thực hiện các tác vụ sau:

- 6 dòng mã thêm không gian cho các biểu thức biểu tượng vào cấu trúc dữ liệu mô tả các thanh ghi của CPU giả lập.
- 4 dòng mã còn lại chèn các TCG ops vào các lệnh gọi và trả về của máy khách. Điều này là tùy chọn, nhưng cho phép mã mượn từ QSYM duy trì một ngăn xếp gọi bóng (xem Mục III-G). Để hỗ trợ một kiến trúc mục tiêu khác, người ta chỉ cần xác định các phép toán gọi và trả về tương ứng của kiến trúc đó. Việc điều chỉnh chúng cho các kiến trúc CPU khác chỉ là một công việc sao chép và dán đơn giản.

Tác giả đã xác nhận tuyên bố về khả năng thích ứng dễ dàng bằng cách thêm hỗ trợ cho AArch64 vào SymQEMU. Việc này yêu cầu 17 dòng mã C, không tính phần mã công cụ gọi và trả về tùy chọn. Lưu ý rằng việc triển khai hiện tại yêu cầu các kiến trúc máy khách 64-bit (để các địa chỉ máy chủ có thể được truyền trong các thanh ghi máy khách), nhưng không có lý do cơ bản nào cho hạn chế này—nó có thể được loại bỏ với một nỗ lực phát triển một lần.

Tóm lại, SymQEMU có thể chạy trên tất cả các kiến trúc máy chủ có liên quan và hỗ trợ phân tích các tệp nhị phân đã biên dịch cho bất kỳ kiến trúc máy khách nào mà QEMU có thể xử lý, với nỗ lực không đáng kể.

D. So sánh với các thiết kế trước đây

Giống như angr và S2E, SymQEMU tuân theo cách tiếp cận truyền thống trong việc thực hiện xử lý biểu tượng ở cấp độ của một đại diện trung gian, điều này giúp giảm đáng kể độ phức tạp của việc triển khai. Tuy nhiên, khác với hai hệ thống này, SymQEMU thực hiện thực thi biểu tượng dựa trên biên dịch, cho phép nó đạt được hiệu suất cao hơn nhiều.

So với QSYM, lợi thế quan trọng nhất trong thiết kế của SymQEMU là tính linh hoạt về kiến trúc trong khi vẫn duy trì tốc độ thực thi cao. Việc xây dựng trên nền tảng QEMU giúp SymQEMU tận dụng được số lượng lớn các nền tảng mà phần mềm giả lập này hỗ trợ. SymCC, mặc dù không thể phân tích các tệp nhị phân, nhưng cũng chia sẻ phương pháp tiếp cận dựa trên biên dịch với SymQEMU. Cả hai đều chèn xử lý biểu tượng vào chương trình mục tiêu bằng cách thay đổi đại diện trung gian của nó và cả hai đều biên dịch kết quả xuống mã máy để có thể thực thi hiệu quả. Tuy nhiên, SymCC được thiết kế để hoạt động trong một trình biên dịch, trong khi SymQEMU giải quyết các vấn đề khác biệt gặp phải trong phân tích tệp nhị phân mà không có mã nguồn,: trong khi SymCC thực hiện thao tác chèn vào LLVM bitcode trong quá trình biên dịch (dựa trên mã nguồn), SymQEMU thực hiện thao tác chèn vào TCG ops trong quá trình biên dịch động của mã nhị phân. Hơn nữa, SymQEMU giải quyết các vấn đề không tương thích giữa kiến trúc mục tiêu và máy chủ, một vấn đề không xuất hiện trong trường hợp của SymCC vì mã nguồn phần lớn không phụ thuộc vào kiến trúc mục tiêu.

Trong bối cảnh này, Tác giả muốn nhấn mạnh rằng SymQEMU có thể hỗ trợ phân tích chéo kiến trúc, tức là kiến trúc CPU mà chương trình thử nghiệm được biên dịch không cần phải khớp với kiến trúc của máy thực hiện phân tích.

Tác giả tin rằng phương pháp của tác giả kết hợp những ưu điểm chính của angr và S2E (tức là tính độc lập nền tảng) và QSYM (tức là hiệu suất), nhưng tránh được những nhược điểm của chúng (hiệu suất thấp và phụ thuộc vào một kiến trúc cụ thể, tương ứng). Hơn nữa, tác giả đã tìm ra cách áp dụng ý tưởng cốt lõi của SymCC về thực thi biểu tượng dựa trên biên dịch vào các tệp nhị phân.

Bảng 1 tổng hợp sự so sánh. Tốc độ đề cập đến việc tập trung vào tốc độ thực thi, **multiarch** nghĩa là khả năng dễ dàng di chuyển giữa các kiến trúc CPU khác nhau, **binary-only** có nghĩa là hỗ trợ phân tích mà không cần mã nguồn, và **cross-architecture** có nghĩa là khả năng phân tích các chương trình nhắm đến một kiến trúc khác với kiến trúc của máy chủ.

Bảng 1: So sánh giữa SymQEMU với các trình thực thi biểu tượng tiêu biểu

Tên gọi	Ngôn ngữ	Biểu diễn trung gian	Tốc độ	Hỗ trợ đa kiến trúc	Chỉ thực thi tệp nhị phân	Kiến trúc chéo
Angr	Python	VEX	x	v	v	v
S2E	C/C++	TCG & LLVM	x	v	v	v
QSYM	C++	không có	v	x	v	x
SynCC	C++	LLVM	v	v	x	x
SymQEMU	C/C++	TCG	v	v	v	v

E. Quản lý bộ nhớ

Khi SymQEMU thực thi chương trình đang phân tích, nó xây dựng các biểu thức biểu tượng mô tả các kết quả trung gian và các ràng buộc đường đi. Lượng bộ nhớ cần thiết cho những biểu thức này sẽ tăng lên theo thời gian, vì vậy SymQEMU cần một cách để dọn dẹp các biểu thức không còn cần nữa. Trước khi mô tả phương pháp quản lý bộ nhớ của SymQEMU, chúng ta hãy cùng thảo luận lý do tại sao việc quản lý bộ nhớ lại cần thiết ngay từ đầu. Dù sao thì, kết quả trung gian trong bất kỳ chương trình hợp lý nào cũng phải có tác động đến luồng điều khiển hoặc trở thành một phần của kết quả cuối cùng—trong trường hợp trước, các biểu thức tương ứng được thêm vào bộ các ràng buộc đường đi và do đó không thể dọn dẹp, và trong trường hợp sau, các biểu thức trở thành các biểu thức con trong mô tả kết quả cuối cùng. Vậy làm thế nào các biểu thức biểu tượng lại có thể trở nên không cần thiết? Chìa khóa của vấn đề là kết quả chương trình về mặt lý thuyết là một phần của kết quả chương trình, nhưng nó có thể được tạo ra ngay từ khi chưa hoàn tất thực thi. Hãy xem xét ví dụ về một công cụ nén tệp liệt kê nội dung của một tệp lưu trữ, in tên các tệp một cách lần lượt: sau mỗi phần dữ liệu đầu ra được tạo ra, chương trình có thể xóa dữ liệu chuỗi tương ứng, và SymQEMU nên dọn dẹp các biểu thức biểu tượng tương ứng. Nếu không, các biểu thức sẽ tích tụ và, trong trường hợp xấu nhất, sẽ chiếm hết bộ nhớ có sẵn.

Lý tưởng nhất, chúng ta sẽ xóa các biểu thức biểu tượng ngay sau khi chúng được sử dụng lần cuối. QSYM, mà Tác giả tái sử dụng phần backend, sử dụng con trỏ thông minh C++ để đạt được mục tiêu này. Tuy nhiên, chúng ta không thể dễ dàng áp dụng cùng một cách tiếp cận trong phiên bản sửa đổi của QEMU: TCG, thành phần QEMU trung tâm trong cơ chế thực thi của Tác giả, là một trình biên dịch động—vì lý do hiệu suất, nó không thực hiện bất kỳ phân tích mở rộng nào đối với mã đã được dịch (khác với các trình biên dịch tĩnh, thường thu thập một lượng thông tin đáng kể liên quan đến phạm vi và tuổi thọ biến). Điều này khiến việc xác định chính xác vị trí để chèn mã dọn dẹp trong chương trình đã dịch trở nên khó khăn. Hơn nữa, kinh nghiệm cho thấy rằng hầu hết các chương trình chỉ chứa một lượng dữ liệu biểu tượng nhỏ và ít có biểu thức nào trở thành dữ liệu rác trong quá trình thực thi, vì vậy chúng ta không muốn phương pháp dọn dẹp của mình gây ra quá tải đáng kể trong trường hợp thông thường khi tất cả các biểu thức có thể nằm trong bộ nhớ cho đến cuối chương trình.

Tác giả đã chọn một phương pháp dọn dẹp lạc quan dựa trên một bộ thu gom rác biểu thức: SymQEMU theo dõi tất cả các biểu thức biểu tượng thu được từ backend, và nếu số lượng của chúng trở nên quá lớn, nó sẽ kích hoạt một lần thu gom. Quan sát cốt lõi là tất cả các biểu thức sống có thể được tìm thấy bằng cách quét (1) các thanh ghi biểu tượng của CPU giả lập và (2) các vùng bóng trong bộ nhớ lưu trữ các biểu thức biểu tượng tương ứng với nội dung bộ nhớ biểu tượng; cả hai đều được backend biết đến. Sau khi liệt kê tất cả các biểu thức sống, SymQEMU có thể so sánh bộ các biểu thức này với bộ tất cả các biểu thức từng được tạo ra và giải phóng những biểu thức không còn sống nữa. Đặc biệt, khi một chương trình loại bỏ kết quả của một phép tính khỏi thanh ghi và bộ nhớ (như trong ví dụ về công cụ nén trên), các biểu thức tương ứng không còn được coi là sống nữa và do đó sẽ được giải phóng. Tác giả đã kết nối bộ thu gom rác biểu thức với cơ chế quản lý bộ nhớ dựa trên con trỏ thông minh của QSYM—cả hai cơ chế này cần phải đồng ý rằng một biểu thức không còn được sử dụng trước khi có thể được giải phóng.

F. Sửa đổi các hoạt động của TCG:

Cách tiếp cận của Tác giả về cơ bản yêu cầu khả năng chèn các lệnh mới vào danh sách các thao tác (ops) của TCG, danh sách này đại diện cho một đoạn mã mục tiêu. Tuy nhiên, TCG chưa bao giờ được thiết kế để cho phép thực hiện các sửa đổi sâu rộng như vậy trong quá trình dịch mã—vì là một bộ dịch động, nó tập trung mạnh vào tốc độ. Do đó, có rất ít hỗ trợ cho việc chỉnh sửa chương trình đối với các thao tác của TCG. Ngược lại, LLVM, chẳng hạn, cung cấp một API phong phú để các bước xử lý của trình biên dịch có thể kiểm tra và sửa đổi mã bitcode của LLVM. TCG chỉ đơn giản lưu trữ các lệnh trong một danh sách liên kết phẳng, không có bất kỳ cấu trúc cấp cao nào có thể điều hướng được, chẳng hạn như các khối cơ bản (basic blocks). Hơn nữa, luồng điều khiển được kỳ vọng là tuyến tính trong một khối dịch thuật (với rất ít ngoại lệ), khiến cho các tối ưu hóa như kiểm tra tính cụ thể tích hợp của SymCC trở nên khó khăn.

Để giảm thiểu xung đột với cơ sở hạ tầng của TCG, việc triển khai của Tác giả tạo ra xử lý biểu tượng cho từng lệnh mục tiêu ngay khi chính lệnh đó được tạo ra. Dù cách này tránh được các vấn đề với bộ tối ưu hóa và trình tạo mã của TCG, nhưng nó làm cho các tối ưu hóa tĩnh nâng cao trở nên bất khả thi vì Tác giả chỉ có thể xem xét một lệnh tại một thời điểm. Đặc biệt, Tác giả có rất ít cơ hội để xác định một cách tĩnh liệu một giá trị tạm thời cụ thể có phải là giá trị cụ thể hay không. Tương tự, Tác giả không thể tạo ra các bước nhảy

bỏ qua trực tiếp các tính toán biểu tượng nếu tất cả các toán hạng hóa ra đều cụ thể tại thời điểm chạy. Thay vào đó, Tác giả chấp nhận một thỏa hiệp để phù hợp với các ràng buộc của môi trường hoạt động của TCG (đặc biệt là nhu cầu dịch động nhanh), đồng thời vẫn cho phép đạt được tốc độ thực thi tương đối cao: Tác giả thực hiện các kiểm tra tính cụ thể trong thư viện hỗ trợ—cách này cho phép Tác giả vẫn có thể bỏ qua các tính toán biểu tượng khi các đầu vào là cụ thể, nhưng kiểm tra này sẽ tốn thêm một cuộc gọi tới thư viện.

IV. ĐÁNH GIÁ:

Để đánh giá SymQEMU, Tác giả đã thực hiện ba nhóm thí nghiệm khác nhau:

- Tác giả so sánh nó với một số công cụ fuzzing tiên tiến nhất hiện nay với sự hỗ trợ của Google FuzzBench.
- Vì FuzzBench không bao gồm các công cụ thực thi biểu tượng, Tác giả đã tiến hành so sánh với các trình thực thi biểu tượng chỉ nhị phân phổ biến trên một tập hợp các chương trình thực tế.
- Để đánh giá sự khác biệt về tốc độ thực thi giữa SymQEMU, QSYM và SymCC, Tác giả đã thực hiện một so sánh hiệu năng giữa các trình thực thi kết hợp này trên các đầu vào cố định.

A. FuzzBench

Google đã công bố FuzzBench vào tháng 3 năm 2020 như là “một dịch vụ tự động hóa hoàn toàn, mã nguồn mở và miễn phí để đánh giá các công cụ fuzzing”.

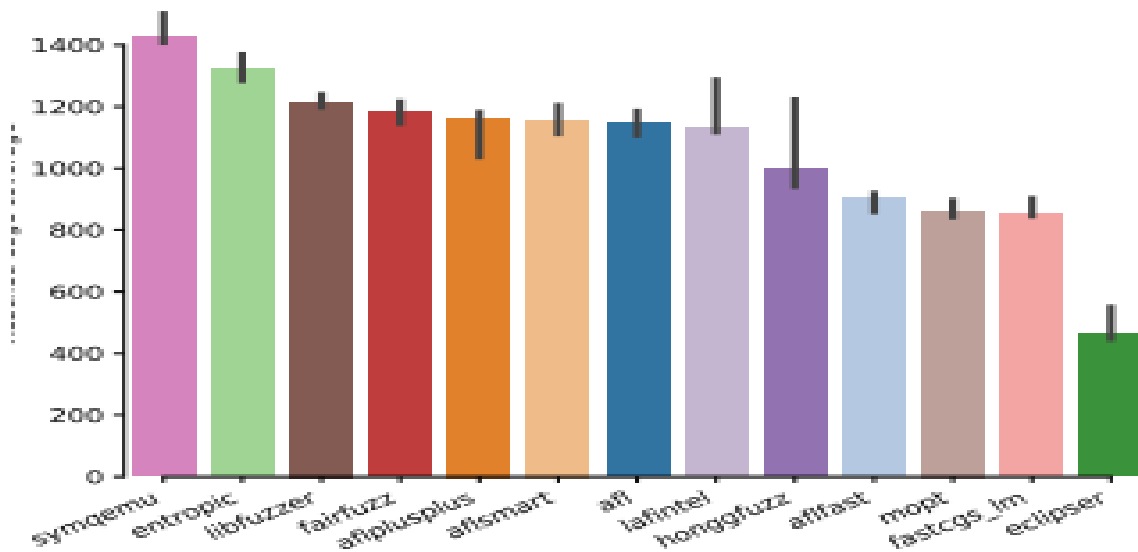
Công cụ này kiểm tra các trình fuzzing trong một môi trường được kiểm soát, so sánh hiệu suất của chúng trên một số lượng lớn mục tiêu từ Google OSS-Fuzz, bộ sưu tập các mục tiêu fuzz dành cho phần mềm mã nguồn mở.

Thí nghiệm

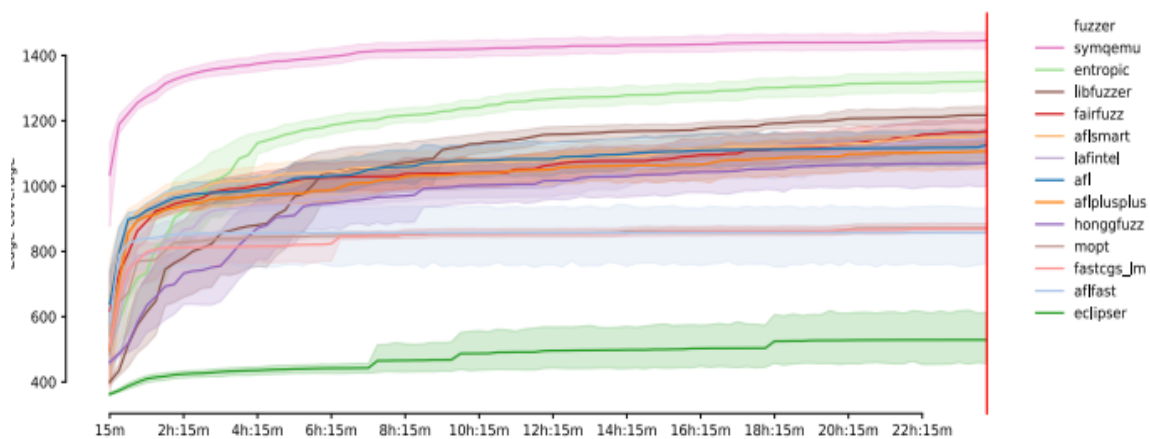
- Mỗi mục tiêu được đánh giá dựa trên edge coverage (phạm vi bao phủ mã) đạt được bởi các công cụ fuzzing.
- Để tích hợp một công cụ phân tích mới, chỉ cần cấu hình một Docker container để thiết lập môi trường, biên dịch các chương trình mục tiêu, và khởi chạy phân tích.
- Tác giả đã thêm một sự kết hợp giữa SymQEMU và AFL vào tập hợp các công cụ phân tích, và nhóm FuzzBench đã thực hiện các thí nghiệm.
- Trong tổng cộng 24 giờ, SymQEMU và 12 cấu hình fuzzing khác đã được thử nghiệm trên 21 mục tiêu, mỗi mục tiêu được thử nghiệm 15 lần (tương đương với khoảng 10 năm sử dụng lõi CPU).

Kết quả

- Kết quả chi tiết được thể hiện trong Hình 7, 8, 9 và 10 đối với hai mục tiêu cụ thể, và Bảng II tóm tắt các kết quả.
- Trung bình trên tất cả các thí nghiệm, SymQEMU vượt qua tất cả các công cụ fuzzing, ngoại trừ Honggfuzz, trong đó 5 công cụ bị vượt qua với độ tin cậy thống kê, bao gồm cả công cụ mạnh mẽ phổ biến libfuzzer.
- Trên 3/21 tiêu chí, SymQEMU đạt được coverage cao nhất trong tất cả các công cụ.
- SymQEMU vượt qua pure AFL trên 14 tiêu chí, nhưng pure AFL vẫn hoạt động tốt hơn trên 7 tiêu chí khác.



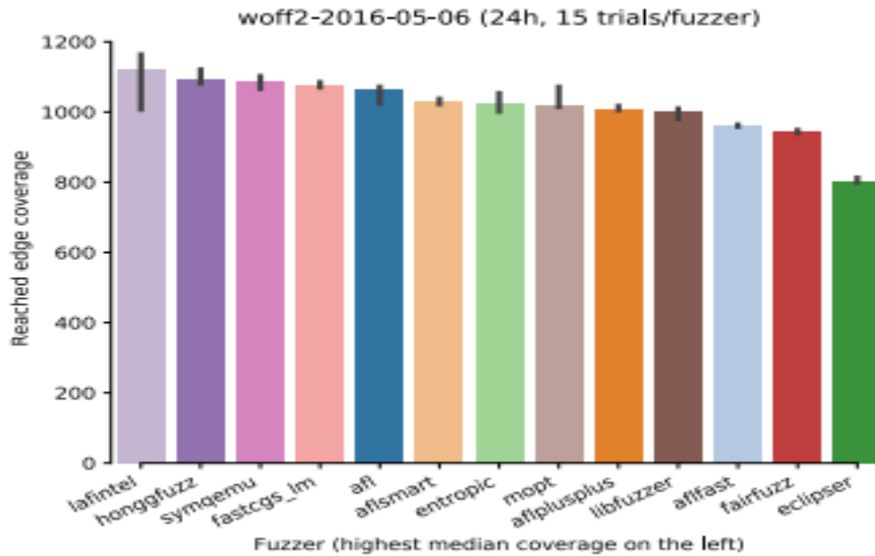
Hình 7. Trích đoạn từ báo cáo FuzzBench: Xếp hạng theo mức độ bao phủ trung bình đạt được đối với mục tiêu FuzzBench lcms. SymQEMU vượt qua tất cả các công cụ khác trên mục tiêu này.



Hình 8. Trích đoạn từ báo cáo FuzzBench: Tăng trưởng bao phủ trung bình theo thời gian (và khoảng tin cậy 95%) đối với mục tiêu FuzzBench lcms. SymQEMU vượt qua tất cả các công cụ khác trên mục tiêu này.

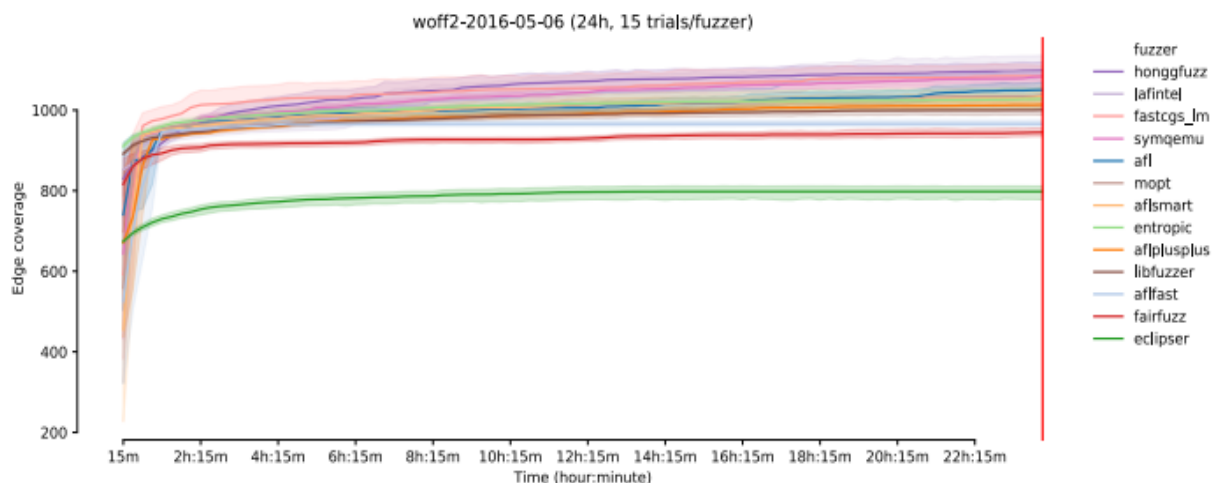
Phân tích

- Hiệu quả của thực thi biểu tượng phụ thuộc vào nhiều yếu tố:
 - Tập dữ liệu ban đầu hoặc từ điển có sẵn.
 - Đặc điểm của mã mục tiêu, như việc sử dụng nhiều hàm hash hoặc các thao tác không thể đảo ngược, thường làm giảm hiệu quả của thực thi biểu tượng.
- SymQEMU đạt được kết quả này mà không sử dụng mã nguồn của các mục tiêu, điều này đặc biệt ấn tượng.
- Đa số các mục tiêu đều đi kèm với các seed corpus hoặc dictionary chất lượng tốt, dẫn đến việc thực thi biểu tượng không đóng góp nhiều vào coverage khi đã có các seed sẵn có.



Hình 9. Trích từ báo cáo FuzzBench:

Xếp hạng theo mức độ phủ đạt được trung vị cho mục tiêu FuzzBench woff2. SymQEMU đạt vị trí thứ 3 trên mục tiêu này.



Hình 10. Trích từ báo cáo FuzzBench:

Tăng trưởng mức độ phủ trung bình theo thời gian (và khoảng tin cậy 95%) cho mục tiêu FuzzBench woff2. SymQEMU đạt vị trí thứ 3 trên mục tiêu này.

Hướng cải tiến

- Hiện tại, việc tích hợp chỉ định một phần cố định thời gian CPU cho thực thi biểu tượng.
- Tác giả tin rằng chiến lược phối hợp tốt hơn giữa trình fuzzing và công cụ thực thi biểu tượng (như ý tưởng của công cụ Pangolin gần đây) có thể cải thiện kết quả hơn nữa.

B. So sánh với các hệ thống thực thi biểu tượng khác:

Mục tiêu chính của SymQEMU là thực thi biểu tượng chỉ trên nhị phân. Trong phần này, Tác giả so sánh SymQEMU với các công cụ tiên tiến trong lĩnh vực này. Cụ thể, Tác giả so sánh SymQEMU với S2E vì giống như SymQEMU, S2E cũng dựa trên QEMU, và với QSYM vì nó là công cụ thực thi biểu tượng trên nhị phân nhanh nhất mà Tác giả biết. Tác giả đã loại trừ angr khỏi so sánh vì các thí nghiệm sơ bộ cho thấy tốc độ thực thi của nó thấp hơn đáng kể so với các công cụ khác; angr ưu tiên tính linh hoạt và sự dễ dàng khi sử dụng tương tác hơn là tốc độ thô. Cuối cùng, tác giả thêm AFL thuần túy như một baseline và so sánh với SymCC vì SymCC giới thiệu khái niệm thực thi biểu tượng dựa trên biên dịch. Tuy nhiên, cần lưu ý rằng SymCC có lợi thế so với các công cụ khác vì nó sử dụng mã nguồn của chương trình cần thử nghiệm, ví dụ, nó có thể tận dụng cấu trúc mã cấp cao và tối ưu hóa của trình biên dịch. Tuy nhiên, tác giả chỉ có thể đánh giá SymCC đối với các mục tiêu mã nguồn mở.

Để thực hiện so sánh, Tác giả thực hiện fuzzing kết hợp của một số chương trình mục tiêu và đo lường độ phủ mã theo thời gian. Tác giả sử dụng khái niệm độ phủ của AFL vì lý do tương tự như trong đánh giá SymCC: đó là yếu tố thúc đẩy quá trình khám phá của AFL. Theo các khuyến nghị của Klees, tác giả phân tích mỗi mục tiêu trong 24 giờ và lặp lại mỗi thí nghiệm 30 lần. Để kiểm tra tính có ý nghĩa thống kê, Tác giả sử dụng kiểm định Mann-Whitney U hai đuôi, cũng như đã được Klees khuyến nghị. Các mục tiêu của Tác giả bao gồm các chương trình mã nguồn mở như OpenJPEG, libarchive, và tcpdump, cùng với chương trình mã nguồn đóng rar. Lý do Tác giả chọn những chương trình này là tác giả đã sử dụng ba công cụ mã nguồn mở này trước đó để đánh giá SymCC, vì vậy Tác giả biết rằng cả SymCC và QSYM đều hoạt động trên chúng, và rar là một chương trình mã nguồn đóng dễ thu thập và có yêu cầu chặt chẽ về định dạng đầu vào, điều này tạo ra các thử thách thú vị cho thực thi biểu tượng, và giấy phép của nó không cấm loại phân tích này. Đối với OpenJPEG và rar, Tác giả cung cấp một đầu vào seed theo định dạng mong đợi; còn đối với libarchive và tcpdump, Tác giả bắt đầu với một corpus rỗng.

Các hệ thống so sánh được thiết lập như sau:

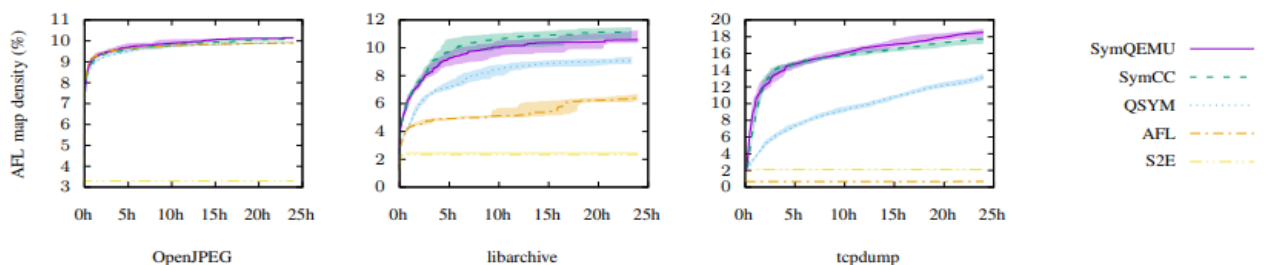
- SymQEMU, QSYM và SymCC: Tác giả chạy các hệ thống này cùng với AFL, sử dụng cùng một tích hợp như trong các công bố về QSYM và SymCC (tức là trao đổi các test case qua hàng đợi fuzzer trong chế độ phân tán của AFL). Tác giả thực thi một phiên bản chính AFL, một phiên bản phụ AFL, và một phiên bản SymQEMU, QSYM hoặc SymCC, mỗi công cụ chạy trên một lõi CPU với 2 GB RAM. AFL được phép sử dụng mã nguồn khi có sẵn; nếu không, Tác giả chạy nó trong chế độ QEMU.
- S2E: Đối với S2E, Tác giả tạo một dự án phân tích cho mỗi mục tiêu, làm cho đầu vào thử nghiệm hoàn toàn là biểu tượng khi có, và cung cấp một tệp biểu tượng tắt cả các giá trị bằng không nếu không có. Tác giả kích hoạt plugin FunctionModels và mở rộng plugin TestCaseGenerator để tạo ra một test case mới mỗi khi trạng thái thực thi mới được fork. Tác giả sử dụng ngăn xếp tìm kiếm mặc định và chạy thí nghiệm trong ảnh Debian 64-bit do các tác giả của S2E cung cấp. Vì chế độ song song của S2E không ổn định trong các thí nghiệm của Tác giả, Tác giả tổng hợp kết quả từ ba phân tích độc lập (để phù hợp với ba lõi CPU mà SymQEMU, QSYM và

SymCC có sẵn); chi tiết có trong Phụ lục A. Để đánh giá coverage mã, Tác giả sử dụng AFL sau khi kết thúc phân tích.

- AFL thuần túy: Tác giả thực thi AFL trong chế độ phân tán, chạy một phiên bản chính và hai phiên bản phụ, mỗi phiên bản trên một lõi CPU với 2 GB RAM. Giống như đối với SymQEMU, QSYM và SymCC, Tác giả cung cấp cho AFL quyền truy cập vào mã nguồn của mục tiêu khi có sẵn và sử dụng chế độ QEMU nếu không có mã nguồn.

Các thí nghiệm được thực hiện trên bộ vi xử lý Intel Xeon Platinum 8260. Tổng cộng, Tác giả đã dành khoảng 5 năm CPU (4 chương trình mục tiêu, 5 hệ thống so sánh, 3 lõi mỗi thí nghiệm, 30 lần lặp lại, 24 giờ).

Hình 11 hiển thị kết quả cho các mục tiêu mã nguồn mở. Tác giả thu thập dữ liệu độ phủ cho AFL và các fuzzer lai từ các nhật ký được ghi bởi afl-fuzz, sử dụng cùng một bộ nhị phân AFL-instrumented để đánh giá mỗi công cụ; đối với S2E, Tác giả chạy các đầu vào chương trình được tạo ra qua afl-showmap (một lần nữa, sử dụng cùng các tệp nhị phân) để tính toán chỉ số độ phủ tương đương. Hơn nữa, xin lưu ý rằng tác giả đã sử dụng các chiến lược giống nhau để tích hợp AFL với QSYM, SymCC và SymQEMU. Tác giả nhận thấy rằng SymQEMU đạt được độ phủ mã tốt hơn đáng kể theo thời gian so với cả QSYM và S2E, do đó vượt trội hơn những công cụ thực thi biểu tượng mã nhị phân tiên tiến này. Nó cũng bao phủ nhiều mã hơn cả AFL thuần túy, cho thấy giá trị của việc thực thi biểu tượng trong việc khám phá các chương trình mục tiêu. Cuối cùng, SymQEMU, một cách ngạc nhiên, đạt được mức độ phủ mã tương đương với kết quả của SymCC, mặc dù SymCC có quyền truy cập vào mã nguồn của các mục tiêu và do đó có nhiều tiềm năng tối ưu hóa hơn. Điều tra thủ công cho thấy SymCC không sử dụng tiềm năng này đến mức tối đa; ví dụ, nó không kích hoạt một lần tối ưu hóa bộ nhớ-to-register sau khi chèn công cụ đo lường của nó (dẫn đến các thao tác bộ nhớ không cần thiết trong chương trình mục tiêu), cũng như không sử dụng tối ưu hóa liên kết thời gian để in-line các cuộc gọi tới thư viện hỗ trợ. Tác giả tin rằng đây là lý do chính khiến một công cụ thực thi biểu tượng mã nhị phân như SymQEMU có thể giữ được mức độ tương đương với một công cụ dựa trên mã nguồn như SymCC. Tóm lại, kết quả xác nhận rằng SymQEMU hiệu quả hơn các hệ thống thực thi biểu tượng mã nhị phân khác trong so sánh của Tác giả.

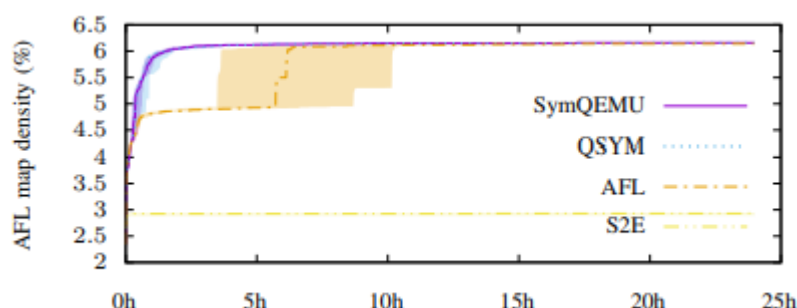


Hình 11. Mức độ phủ theo thời gian trên các mục tiêu mã nguồn mở:

Đồ thị thể hiện mật độ bản đồ độ phủ của AFL, với giá trị trung vị và hành lang tin cậy 95%.

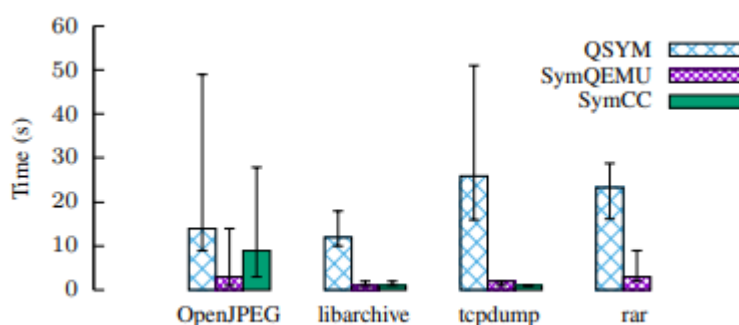
SymQEMU đạt mức độ phủ cao hơn tất cả các hệ thống khác với ý nghĩa thống kê (kiểm định Mann-Whitney U, $p < 0.005$ hai phía), ngoại trừ trên libarchive, nơi không có sự khác biệt đáng kể về mật độ thống kê so với SymCC. Tuy nhiên, cần lưu ý rằng SymCC yêu cầu mã nguồn của chương trình được kiểm thử.

Trong phân tích của Tác giả về libarchive, SymQEMU đã tìm thấy một đầu vào dẫn đến lỗi use-after-free trên heap. Lỗi này có thể được kích hoạt, ví dụ, bằng cách yêu cầu người dùng liệt kê nội dung của một kho lưu trữ đã được thao tác bằng tiện ích bsdtar, và tác giả cho rằng có khả năng nó có thể được khai thác. Tác giả đã báo cáo sự cố này cho các nhà phát triển libarchive; tại thời điểm viết bài, Tác giả chưa nhận được phản hồi.



Hình 12. Mức độ phủ theo thời gian trên chương trình mã nguồn đóng rar

Đồ thị thể hiện mật độ bản đồ độ phủ của AFL, với giá trị trung vị và hành lang tin cậy 95%. Tất cả các công cụ ngoại trừ S2E đều hội tụ về cùng một mức độ phủ, nhưng SymQEMU đạt được mức này nhanh hơn AFL, do đó cần ít tài nguyên tính toán hơn trên mỗi mức độ phủ. Hơn nữa, tốc độ của SymQEMU tương tự như QSYM, nhưng QSYM khó được chuyển từ x86 sang các mục tiêu khác.



Hình 13. Thời gian thực thi của các chương trình mục tiêu trên từng bộ thực thi biểu tượng.

Biểu đồ so sánh thời gian thực thi của các chương trình mục tiêu khi sử dụng các bộ thực thi biểu tượng khác nhau. Kết quả cho thấy SymQEMU nhanh hơn QSYM và ít nhất ngang bằng về tốc độ với SymCC dựa trên mã nguồn. Lưu ý rằng khái niệm về thời gian thực thi không áp dụng cho S2E, và SymCC không thể phân tích chương trình rar do không có mã nguồn.

Hình 12 hiển thị kết quả cho chương trình rar mã nguồn đóng. SymQEMU, QSYM và AFL đều hội tụ về cùng một mức độ phủ mã, nhưng SymQEMU đạt đến mức bão hòa nhanh như QSYM ít linh hoạt về kiến trúc và nhanh hơn AFL. Lưu ý thêm rằng SymQEMU và QSYM nhanh chóng phát hiện các đường dẫn mà AFL thuần túy (tức là không sử dụng thực thi biểu tượng) cần thêm thời gian để tìm ra. S2E không thể phân tích nhiều mã như các công cụ khác nhưng có thể bao phủ nhiều không gian dữ liệu hơn trên các đường dẫn đã được phát hiện. Thí nghiệm này cho thấy SymQEMU có thể làm việc với các mục tiêu mã nguồn đóng, giống như các công cụ thực thi biểu tượng mã nhị phân khác, nhưng với lợi thế bổ sung là hỗ trợ một số lượng lớn các kiến trúc mục tiêu.

Điều thú vị là thực thi biểu tượng thường đóng góp nhiều nhất vào độ phủ mã khi không có đầu vào hạt giống, như được chứng minh trong phân tích của Tác giả về libarchive và tcpdump. Trong khi đó, trên OpenJPEG và rar, các tệp hạt giống cung cấp cho AFL đủ thông tin để cũng đạt được mức độ phủ mã tốt trong một khoảng thời gian tương đối ngắn.

Cuối cùng, Hình 13 hiển thị thời gian thực thi của các công cụ thực thi biểu tượng trong các thí nghiệm của Tác giả, cung cấp bằng chứng rằng SymQEMU luôn nhanh hơn QSYM và ít nhất cũng đạt được mức tương đương với SymCC dựa trên mã nguồn. Tác giả đã bỏ qua S2E trong hình vì không có khái niệm tương đương về thời gian thực thi đối với phương pháp của nó: trong khi Tác giả có thể đo thời gian tồn tại của mỗi trạng thái thực thi, điều này sẽ bỏ qua thực tế là S2E thực hiện nhiều kiểm tra mà các hệ thống khác ủy thác cho fuzzer và các công cụ kiểm tra, và do đó sẽ đặt S2E vào thế bất lợi trong so sánh.

C. So sánh benchmark

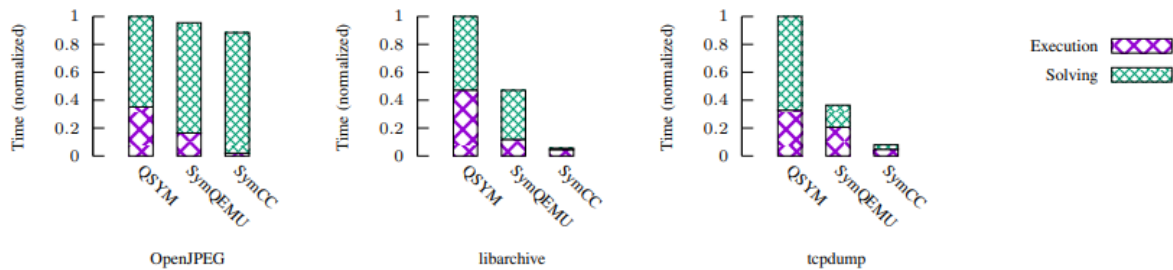
Chúng ta đã thấy rằng SymQEMU vượt trội hơn các công cụ thực thi biểu tượng mã nhị phân tiên tiến trong việc fuzzing lai thực tế. Bây giờ, chúng ta sẽ kiểm tra giả thuyết rằng những kết quả đó thực sự là nhờ vào tốc độ thực thi cao của SymQEMU. Để làm điều này, Tác giả đã thực hiện một bộ thí nghiệm thứ ba với mục tiêu đánh giá chính xác tốc độ thực thi của SymQEMU so với hai công cụ concolic thực thi còn lại trong so sánh của Tác giả, là SymCC và QSYM.

Ý tưởng chính của thí nghiệm này là thực hiện thực thi concolic trên một bộ các đầu vào cố định, do đó làm cho tất cả các hệ thống trong so sánh theo cùng một đường dẫn trên cùng các chương trình mục tiêu. Nói cách khác, Tác giả loại bỏ một yếu tố biến từ so sánh: lựa chọn các đường dẫn cần theo. Tác giả kết hợp các trường hợp thử nghiệm mà SymQEMU, SymCC và QSYM tìm thấy trong phiên fuzzing lai kéo dài 24 giờ; sau đó, Tác giả chọn ngẫu nhiên 1000 trường hợp thử nghiệm cho mỗi mục tiêu. Tác giả thực hiện thực thi concolic trên các đầu vào đã chọn, đo thời gian chi tiêu trong việc thực thi và giải quyết SMT, tương ứng.

Hình 14 hiển thị sự phân chia thời gian quan sát được cho mỗi mục tiêu và công cụ thực thi biểu tượng. Tác giả thấy rằng, trên cả ba mục tiêu, SymQEMU dành ít thời gian hơn trong việc thực thi so với QSYM; điều này cung cấp bằng chứng rằng hiệu suất cao của SymQEMU trong fuzzing lai (xem Hình 11) thực sự là nhờ vào tốc độ thực thi cao hơn. SymCC, công cụ dựa trên mã nguồn, dành ít thời gian hơn trong việc thực thi so với cả SymQEMU và QSYM vì, khác với các công cụ thực thi biểu tượng mã nhị phân, SymCC không gặp phải chi phí của việc biên dịch nhị phân động hoặc đo lường nhị phân động.

Điều thú vị là ba hệ thống dành những lượng thời gian khác nhau cho việc giải quyết SMT. Vì các đường dẫn chương trình đã được cố định và các công cụ thực thi biểu tượng sử dụng cùng một backend để tương tác với bộ giải, Tác giả kết luận rằng có sự khác biệt trong độ khó của các truy vấn SMT. Kiểm tra thủ công xác nhận rằng các truy vấn của SymCC ngắn hơn và ít lồng nhau hơn so với những truy vấn được tạo ra bởi các hệ thống khác (ngoại trừ mục tiêu OpenJPEG, nơi Tác giả thấy rất nhiều phép toán số học và phép toán cấp bit trong các truy vấn của tất cả các hệ thống, điều này Tác giả cho là do thuật toán nén của định dạng JPEG). Sự khác biệt về độ khó này có thể là do các biểu diễn trung gian

khác nhau mà các phân tích dựa vào. Cụ thể, quan sát của Tác giả rằng SymCC thường tạo ra các truy vấn đơn giản hơn và do đó dành ít thời gian hơn trong bộ giải SMT so với hai hệ thống còn lại cung cấp bằng chứng cho giả thuyết trước đây của Tác giả rằng các biểu diễn trung gian cấp cao hơn dẫn đến các truy vấn SMT đơn giản hơn.



Hình 14. Thời gian dành cho thực thi và giải SMT, trung bình trên việc thực thi concolic một tập hợp cố định các trường hợp kiểm tra (1000 trường hợp mỗi mục tiêu, được chọn ngẫu nhiên và phân tích trên từng bộ thực thi biểu tượng).

Tóm lại, Tác giả đã chỉ ra rằng SymQEMU vượt trội hơn các công cụ thực thi biểu tượng mã nhị phân tiên tiến trong fuzzing lai thực tế, và lý do cho hiệu suất cao của nó là thành phần thực thi nhanh chóng. So với QSYM, SymQEMU đạt được độ phủ mã cao hơn 19% sau 24 giờ (Hình 11 và 12) và thực thi nhanh hơn 58% trong thí nghiệm benchmark (Hình 14).

V. Kết luận:

SymQEMU đã chứng minh được khả năng vượt trội trong việc phát hiện lỗi và độ phủ mã so với các công cụ fuzzing khác. Kết quả nghiên cứu cho thấy rằng việc sử dụng SymQEMU có thể cải thiện đáng kể hiệu quả của quy trình kiểm tra bảo mật, đồng thời giảm thiểu thời gian và tài nguyên cần thiết. Những vấn đề nêu ra trong nghiên cứu cũng chỉ ra rằng có nhiều cơ hội để cải thiện và tối ưu hóa các công cụ fuzzing hiện tại, từ đó nâng cao khả năng phát hiện lỗi và bảo mật cho các ứng dụng thực tế. SymQEMU không chỉ vượt trội hơn so với các công cụ thực thi biểu tượng nhị phân khác mà còn có khả năng hoạt động độc lập với kiến trúc, cho phép nó phân tích các nhị phân từ nhiều nền tảng khác nhau mà không cần điều chỉnh mã nguồn. Điều này làm tăng tính linh hoạt và khả năng ứng dụng của SymQEMU trong các môi trường đa dạng, từ các ứng dụng nhúng đến các hệ thống phức tạp hơn. Kết quả nghiên cứu cho thấy SymQEMU không chỉ đạt được độ phủ mã cao hơn mà còn có thời gian thực thi nhanh hơn, khẳng định giá trị của nó trong lĩnh vực phân tích bảo mật. Tính độc lập kiến trúc của SymQEMU mở ra nhiều cơ hội cho việc phát hiện lỗi và cải thiện an ninh cho các phần mềm mà không cần phụ thuộc vào mã nguồn, điều này đặc biệt quan trọng trong bối cảnh ngày càng nhiều phần mềm được phát triển dưới dạng nhị phân.

Vì phần thực nghiệm, dù em đã cố gắng thử cài đặt và kiểm thử với SymQEMU nhưng để đạt được kết quả so sánh muốn có thì vẫn nằm ngoài khả năng của em. Nên nhóm em xin phép lấy lại số liệu từ thí nghiệm của các tác giả trong bài báo.

VI. Phụ Lục:

Câu hỏi số 1: Những cái instruction của hệ máy gốc không có, tcg làm kiểu gì?

TCG (Tiny Code Generator) là một phần không thể thiếu của hệ thống QEMU, có nhiệm vụ dịch mã máy của chương trình mục tiêu sang các lệnh TCG. Khi một lệnh không được hỗ trợ bởi TCG, QEMU có thể sử dụng các cơ chế như dịch ngược (emulation) hoặc bỏ qua lệnh đó, tùy thuộc vào cách mà TCG được cấu hình. Điều này cho phép QEMU xử lý các lệnh không có sẵn trong hệ máy gốc bằng cách sử dụng các phương pháp thay thế hoặc thông qua việc thực thi mã máy một cách trực tiếp nếu có thể. Vì vậy, những lệnh không có sẵn trên máy gốc, TCG có thể giúp hệ thống xử lý được.

Câu hỏi số 2: Binary cụ thể có giá trị cụ thể trong instrument tcg ảnh hưởng được như thế nào đến tcg?

Khi một binary cụ thể được đưa vào TCG, các giá trị cụ thể trong binary đó có thể ảnh hưởng đến cách mà TCG thực hiện dịch mã và tối ưu hóa. TCG sẽ chuyển đổi các lệnh từ binary sang các lệnh TCG, và trong quá trình này, các giá trị cụ thể có thể được sử dụng để tối ưu hóa việc thực thi, chẳng hạn như thông qua việc sử dụng các kỹ thuật như inline caching hoặc các tối ưu hóa khác dựa trên các giá trị cụ thể của biến trong chương trình. Điều này có thể dẫn đến hiệu suất tốt hơn trong quá trình thực thi mã. Vì sẵn thì SymQEMU là một mã nguồn mở nên việc “custom” thêm vào hệ thống sẽ giúp người dùng kiểm thử được đúng mục đích của mình và đạt hiệu quả tốt hơn.

Câu hỏi số 3: Chương trình cần phân tích chạy trên hệ điều hành khác chứ không phải bản thân công cụ?

Với SymQEMU, chương trình cần phân tích có thể chạy trên một hệ điều hành khác so với hệ điều hành mà công cụ (QEMU) đang chạy. SymQEMU sử dụng cơ chế emulation của QEMU để thực thi mã của chương trình mục tiêu trong không gian người dùng, trong khi các hệ thống gọi (system calls) được dịch để tương thích với kiến trúc của máy chủ. Điều này cho phép SymQEMU thực hiện phân tích mà không cần phải chuẩn bị hệ điều hành cho mỗi kiến trúc mục tiêu, từ đó tiết kiệm thời gian và công sức cho đối tượng muốn kiểm thử. Vì vậy, SymQEMU đã chứng minh được tính độc lập về kiến trúc mạnh mẽ của mình

VII. Nguồn tham khảo:

- Sebastian Poeplau, Aurélien Francillon. SymQEMU: Compilation-based symbolic execution for binaries. NDSS 2021, Network and Distributed System Security Symposium, Feb 2021, San Diego (virtuel), United States.
- F. Bellard, “QEMU, a fast and portable dynamic translator,” in USENIX Annual Technical Conference, FREENIX Track, vol. 41, 2005, p. 46.
- H. Huang, P. Yao, R. Wu, Q. Shi, and C. Zhang, “Pangolin: Incremental hybrid fuzzing with polyhedral path abstraction,” in 2020 IEEE Symposium on Security and Privacy (SP). Los Alamitos, CA, USA: IEEE Computer Society, May 2020, pp. 1613–1627. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/SP40000.2020.00063>
- V. Chipounov, V. Kuznetsov, and G. Candea, “S2E: A platform for in-vivo multi-path analysis of software systems,” in ACM SIGARCH Computer Architecture News, vol. 39, no. 1. ACM, 2011, pp. 265–278.
- Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel et al., “Sok: (state of) the art of war: Offensive techniques in binary analysis,” in 2016 IEEE Symposium on Security and Privacy (SP). IEEE, 2016, pp. 138–157.
- I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, “QSYM: A practical concolic execution engine tailored for hybrid fuzzing,” in 27th USENIX Security Symposium (USENIX Security 18), 2018, pp. 745–761.

VIII. Phân công công việc:

Họ và tên	Vai trò	Phân công công việc	Dung lượng công việc
Trần Thọ Mạnh	Nhóm trưởng	Viết sách	40%
Nguyễn Việt Hưng	Thành viên	Viết docs thuyết trình, biện luận	20%
Nguyễn Tiến Mạnh	Thành viên	Làm slide, biện luận	20%
Nguyễn Khắc Nam Huy	Thành viên	Làm slide, biện luận, thuyết trình	20%

Cam đoan

Nhóm em xin cam đoan không sử dụng bất kì một công cụ nào hay một mô hình trí tuệ nhân tạo nào trong quá trình viết báo cáo

Chữ ký xác nhận của từng thành viên

Nhóm trưởng

Thành viên

Thành viên

Thành viên